Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

CE-1113 Sistemas Empotrados

Documento de Aprendizaje Continuo (AC)

Proyecto I: Sistema embebido a la medida para un robot aspiradora autónomo con reproducción de audio y control remoto

Profesor

Dr.-Ing. Jeferson González Gómez

Integrantes

Gabriel Bolaños Barboza	2022327511
Yerik Chaves Serrano	2022437493
Emmanuel Esquivel Chavarría	2022312336
Sergio Marín Zuñiga	2018171984

6 de octubre de 2026

Índice

1. Introducción **2**

2. AC1: Necesidades de aprendizaje identificadas **2**

3. AC2: Tecnologías nuevas y emergentes **3**

4. AC3: Estrategias implementadas **4**

4.1. Tutoriales del curso como base 4

4.2. Documentación oficial y lectura del código fuente 4

4.3. Verificación incremental en el hardware 4

4.4. Depuración sistemática basada en evidencia 5

4.5. Aislamiento de fallas con imágenes mínimas 5

4.6. Revisión de artefactos y versionado de imágenes 5

4.7. Pruebas A/B en el hardware 5

4.8. Retorno a la última versión funcional 5

4.9. Descartar causas mecánicas 6

4.10. Organización del tiempo y del trabajo 6

4.11. Registro de decisiones 6

4.12. Consulta con el profesor 6

4.13. Uso crítico de asistentes de IA 6

5. AC4: Evaluación crítica de las estrategias **6**

6. Conclusiones **8**

1. Introducción

El Proyecto I exigió integrar en una sola plataforma varias áreas que el grupo no dominaba al inicio: construcción de sistemas operativos a la medida, control de hardware desde Linux, electrónica de potencia, audio embebido y servicios web. Este documento describe el proceso de aprendizaje según los indicadores AC1 a AC4: las necesidades identificadas, las tecnologías nuevas utilizadas, las estrategias aplicadas y la evaluación crítica de su eficacia.

2. AC1: Necesidades de aprendizaje identificadas

Las necesidades surgieron al contrastar los requerimientos de la especificación con los conocimientos previos del grupo (Tabla 1).

Tabla 1: Necesidades de aprendizaje y su origen en el proyecto.

Área	Necesidad	Origen en el proyecto
Sistemas a la medida	Capas, recetas, `.bbappend`, caché `sstate` y configuración de BitBake.	Imagen mínima con Yocto y receta propia reproducible desde cero.
Compilación cruzada	Toolchain ARM, CMake, CPack y separación de paquetes de ejecución y de desarrollo.	Biblioteca `librobot.so` instalada en la imagen.
Hardware desde Linux	GPIO, PWM por hardware y medición de pulsos con `pigpio`.	Motores, sensores HC-SR04 y LEDs.
Audio en Linux	ALSA, módulos del kernel y parámetros de arranque de la Raspberry Pi.	Reproducción MP3 y sonidos de notificación por el jack de 3.5 mm.
Servicios del sistema	Unidades y targets de `systemd`.	Arranque automático del servidor y del sonido de inicio.
Electrónica de potencia	Aislamiento óptico, puente H, baterías de ion de litio, BMS y rieles independientes.	Nota de seguridad obligatoria para proteger la tarjeta prestada.
Software web embebido	Servidor HTTP/WebSocket en C, JSON-RPC 2.0 y cifrado de credenciales.	Control remoto con autenticación.
Navegación	Odometría y representación del área en una grilla 2D.	Mapa de recorrido en tiempo real.
Recursos compartidos del SoC	Qué periféricos comparten el PWM por hardware, el audio analógico, el SPI y el I2S.	El audio del jack se apagaba en cuanto se movían los motores.
Conectividad inalámbrica en Yocto	Driver `brcmfmac`, firmware, módulos de proveedor, `wpa_supplicant` y `systemd-networkd`.	Control remoto por WiFi con una dirección conocida.

Área	Necesidad	Origen en el proyecto
Validación de artefactos	Comprobar el contenido real de la imagen y de los archivos antes de grabar la tarjeta.	Imágenes que no reflejaban los cambios y archivos de audio vacíos.
Medición de recursos	Lectura de `/proc`, `systemd-analyze` y tiempos de servicios.	Métricas obligatorias de rootfs, arranque, RAM y CPU.
Gestión del proyecto	Git Flow, Conventional Commits y gestión de issues.	Rubro de flujo de trabajo Git (10 %).

Además de conocimientos técnicos, el proyecto exigió dos actitudes. La primera fue depurar de forma sistemática, con evidencia y no por prueba y error. La segunda fue documentar cada decisión. El cambio tecnológico también generó necesidades: Yocto publica una versión cada seis meses y las capas de terceros cambian entre versiones. Por ejemplo, `pigpio` no está disponible en la capa `meta-raspberrypi` de la versión scarthgap, y el kernel 6.6 separa el driver del WiFi de la Raspberry Pi 4 en módulos por proveedor que versiones anteriores no requerían.

3. AC2: Tecnologías nuevas y emergentes

Tabla 2: Tecnologías nuevas para el grupo y su aporte al aprendizaje.

Tecnología	Uso en el proyecto	Aporte al aprendizaje
Yocto Project 5.0 (scarthgap, LTS) y BitBake	Imagen `core-image-minimal` y capa propia `meta-robot`.	Comprensión de cómo se compone un sistema operativo a partir de recetas y dependencias.
`meta-raspberrypi`	Soporte de tarjeta (BSP) para la Raspberry Pi 4.	Diferencia entre la capa del fabricante y las capas propias; uso de `.bbappend` para modificarla sin editarla.
`pigpio`	PWM por software con DMA y lectura del HC-SR04 desde `librobot.so`.	Acceso a periféricos desde espacio de usuario; diferencia entre PWM por hardware y por DMA y sus conflictos con otros periféricos.
`systemd`	Servicios de arranque sin interfaz gráfica.	Unidades, dependencias de arranque y diagnóstico con `systemctl` y `journalctl`.
ALSA y `mpg123`	Salida de audio por el jack y control de reproducción.	Funcionamiento de los drivers de audio y de los parámetros de módulos del kernel.
Módulos de proveedor de `brcmfmac` (kernel 6.6)	WiFi de la Raspberry Pi 4 con `kernel-module-brcmfmac-wcc`.	Cómo diagnosticar un driver que carga pero no crea la interfaz, a partir de los mensajes de `dmesg`.

Tecnología	Uso en el proyecto	Aporte al aprendizaje
`wpa_supplicant` y `systemd-networkd`	Conexión al hotspot con IP fija y DHCP.	Configuración declarativa de red en una imagen sin interfaz gráfica.
`ffmpeg` (en el host)	Validación y normalización de volumen de los MP3.	Detección de archivos inválidos antes de integrarlos a la imagen.
Mongoose, WebSocket y JSON-RPC 2.0	Servidor en C y comunicación en tiempo real con el cliente.	Diseño de una API remota sobre un canal bidireccional con bajo consumo de recursos.
Web Crypto API y OpenSSL	Cifrado AES-256-CBC de credenciales.	Uso de primitivas criptográficas estándar en el cliente y el servidor.
Optoacopladores PC817	Separación de las señales entre los GPIO y el L298N.	Diferencia entre separar señales y aislar galvánicamente; además, el optoacoplador invierte la señal, por lo que un PWM de 0 en el GPIO equivale a velocidad máxima.
KiCad	Esquemático del circuito de control.	Captura de esquemáticos y verificación de la asignación de pines.
GitHub Projects	Backlog por épicas e historias de usuario.	Planificación ágil con criterios de aceptación verificables.
Asistentes de IA generativa	Consulta de comandos, recetas y diagnóstico de errores.	Uso crítico de herramientas emergentes, con verificación de cada respuesta sobre el hardware.

4. AC3: Estrategias implementadas

4.1 Tutoriales del curso como base

Antes de trabajar en el proyecto se reprodujeron los tutoriales de Yocto del curso en una máquina virtual. La configuración de `local.conf` del proyecto (`MACHINE`, `rm_work` y `DL_DIR` compartido) parte de ese tutorial. Así el grupo arrancó desde un entorno ya probado.

4.2 Documentación oficial y lectura del código fuente

Cuando la documentación no bastaba, se leyó el código fuente. Por ejemplo, para escribir la receta de `pigpio` se revisó su `CMakeLists.txt`. Así se detectaron tres problemas de empaquetado antes de compilar: bibliotecas sin número de versión, páginas de manual en una ruta no estándar e instalación automática del módulo de Python.

4.3 Verificación incremental en el hardware

Cada avance se probó en la Raspberry Pi antes del siguiente. Se escribió el programa `robot-selftest` para probar la biblioteca sin el servidor, y se revisó el manifiesto de la imagen antes de cada grabación de la tarjeta SD. Las salidas de las pruebas se guardaron como evidencia en `docs/metrics`.

4.4 Depuración sistemática basada en evidencia

La salida de audio por el jack es el caso más representativo. En cada paso se usó un comando de diagnóstico antes de proponer una corrección:

1. `aplay -l` no mostraba la tarjeta del jack: la imagen mínima no incluía el módulo `snd-bcm2835`.
2. Con el módulo agregado, `mpg123` reportaba `Found no driver out of [pulse]`: Poky lo compila solo con PulseAudio, que no está en la imagen. Se corrigió con un `.bbappend` que lo compila con ALSA.
3. El parámetro `enable_headphones` seguía en `N`. `modprobe -v` mostró que el firmware agregaba `enable_headphones=0` en la línea de comandos del kernel, lo cual anulaba la configuración. Se corrigió con un `.bbappend` de `rpi-cmdline`.

Con la misma estrategia se resolvió el cierre abrupto de `pigpio` (`libgcc_s.so.1 must be installed for pthread_cancel to work`), declarando `libgcc` como dependencia de ejecución.

Con la misma estrategia se resolvió la conexión WiFi: `ip link` no mostraba `wlan0` y `dmesg` mostró `brcmf_fwvid_request_module: mod=wcc: failed`. El driver y el firmware estaban en la imagen, pero faltaba el módulo de proveedor `brcmfmac-wcc`, que se agregó a la receta `robot-wifi`.

4.5 Aislamiento de fallas con imágenes mínimas

Cuando el audio dejó de sonar en la imagen completa, se construyó una imagen que solo reproducía música, sin `librobot` ni motores. Esto separó dos causas. Primero, los MP3 copiados a la capa pesaban 0 bytes: `mpg123` terminaba cada canción en un segundo sin reportar error. Segundo, con archivos válidos el audio funcionaba hasta que la biblioteca configuraba el PWM por hardware de los motores (`Cannot flush to my output anymore`), porque el jack usa el mismo periférico. La corrección fue pasar los motores a PWM por software.

4.6 Revisión de artefactos y versionado de imágenes

Ante cambios que no se reflejaban en el robot, se revisó el contenido real de los artefactos: el tamaño y tipo de cada archivo de audio (`ls -lh`, `file`) y la biblioteca dentro de la imagen grabada, extrayendo la partición raíz del archivo `.wic`. Además, cada imagen nueva recibió un nombre distinto (por ejemplo, Demo 1 v2) y un mensaje de versión visible al arrancar, para confirmar en la tarjeta cuál imagen se estaba probando.

4.7 Pruebas A/B en el hardware

La lógica del motor de aspiración se invirtió varias veces sin resolverse, porque cada cambio se basaba en una observación anterior. Se resolvió con una sola imagen de prueba que usaba un nivel distinto en cada modo: alto para el botón manual y bajo para el modo autónomo. Una sola prueba mostró qué nivel encendía el motor, y la versión final aplicó ese nivel a ambos modos.

4.8 Retorno a la última versión funcional

Un intento de sincronizar el arranque de los dos motores empeoró el control manual. En lugar de seguir corrigiendo sobre esa versión, se volvió a la última imagen que funcionaba y se agregó un solo cambio a la vez. La revisión posterior del código mostró la causa: la propuesta suponía que un PWM de 0 detiene el motor, cuando por la inversión del optoacoplador lo pone a máxima velocidad.

4.9 Descartar causas mecánicas

Cuando una rueda se detenía al avanzar, antes de modificar el código se probó cada motor por separado con `robot-selftest`. La causa fue suciedad en la rueda, no el software.

4.10 Organización del tiempo y del trabajo

El backlog se dividió en épicas e historias de usuario con criterios de aceptación, en un tablero de GitHub Projects con hitos por iteración. Cada historia se trabajó en su propia rama. El servidor web se desarrolló en paralelo con un robot simulado, para no depender de que el hardware y la imagen estuvieran terminados.

4.11 Registro de decisiones

Las decisiones de hardware (aislamiento, alimentación y chasis) se documentaron como ADR en el repositorio. Esto permitió que todos los integrantes conocieran las razones de cada decisión aunque no hubieran participado en ella.

4.12 Consulta con el profesor

Cuando el diseño de alimentación mostró que los convertidores no aislados unían las tierras de lógica y potencia, el grupo consultó al profesor antes de cambiar componentes. La solución acordada fue un diodo en la tierra del convertidor de la Raspberry Pi para bloquear las corrientes de retorno.

4.13 Uso crítico de asistentes de IA

Se consultaron asistentes de IA generativa para comandos, recetas y diagnóstico de errores. Ninguna respuesta se aceptó sin verificarla en la máquina virtual o en la Raspberry Pi.

5. AC4: Evaluación crítica de las estrategias

Tabla 3: Eficacia de las estrategias de aprendizaje.

Estrategia	Eficacia	Evidencia y limitación
Tutoriales del curso	Alta	Permitieron construir y arrancar la primera imagen del proyecto en la Raspberry Pi. Limitación: el tutorial usa una toolchain de 64 bits, y se asumió lo mismo para el proyecto hasta que `uname -a` en la tarjeta mostró `armv7l` (32 bits).
Documentación y código fuente	Media	Resolvió el empaquetado de `pigpio`. Limitación: la documentación no advierte los cambios entre versiones de las capas; la ausencia de `pigpio` en scarthgap se descubrió solo al consultar las recetas disponibles.

Estrategia	Eficacia	Evidencia y limitación
Verificación incremental	Alta	Detectó los problemas de `libgcc`, PulseAudio y del jack en la tarjeta, cada uno aislado a un solo cambio. Limitación: cada corrección requería compilar y grabar de nuevo la tarjeta SD. Se mitigó probando primero las correcciones manualmente en la Raspberry Pi.
Depuración sistemática	Alta	Cada causa raíz se identificó con un comando de diagnóstico y no por suposición; las correcciones quedaron en la capa y no como cambios manuales.
Organización del trabajo	Media	El tablero y las ramas dieron trazabilidad. Limitación: las dependencias entre historias (servidor, biblioteca y hardware) retrasaron la integración. El robot simulado redujo el impacto, pero la prueba integrada quedó al final del cronograma.
Registro de decisiones	Alta	Los ADR sirvieron de base directa para el Documento de Diseño. Limitación: se redactaron tarde y una historia de hardware estuvo bloqueada a la espera de su ADR.
Consulta con el profesor	Alta	Resolvió un requisito de seguridad con un componente de bajo costo, sin reemplazar los convertidores. Limitación: la consulta se hizo cuando el diseño de alimentación ya estaba definido; hacerla antes habría evitado la incertidumbre.
Asistentes de IA	Media	Aceleraron la escritura de recetas, el diagnóstico del WiFi y del audio, y la revisión de la imagen generada. Limitación: varias propuestas partían de supuestos no verificados (nombres de paquetes, la polaridad del PWM y la lógica de la aspiradora) y empeoraron el sistema hasta que se probaron en el hardware.
Imágenes mínimas	Alta	Separó en dos pruebas causas que en la imagen completa se confundían (archivos vacíos y conflicto del PWM con el audio). Limitación: cada imagen requiere compilar y grabar la tarjeta, por lo que se usó solo cuando el diagnóstico en la imagen completa no fue concluyente.
Revisión de artefactos y versionado	Alta	Detectó los MP3 de 0 bytes y confirmó qué biblioteca contenía cada imagen. Limitación: se adoptó tarde; antes de nombrar las imágenes por versión se perdieron varias pruebas sin saber qué imagen estaba grabada.
Pruebas A/B en el hardware	Alta	Resolvió en una sola prueba la lógica de la aspiradora, después de varios cambios fallidos basados en suposiciones. Limitación: la lógica quedó resuelta en el servidor y falta unificarla en la biblioteca.
Retorno a la última versión funcional	Alta	Recuperó el control manual en una sola compilación. Limitación: se aplicó después de varias iteraciones; aplicarlo antes habría ahorrado tiempo de compilación y pruebas.

Estrategia	Eficacia	Evidencia y limitación
Entorno de construcción en máquina virtual	Media	La caché `sstate` y el directorio de descargas compartido redujeron una reconstrucción incremental a cerca de un minuto. Limitación: el disco se llenó durante la recompilación por el cambio a `systemd` y hubo que ampliarlo. Además, la copia manual de archivos entre la máquina virtual y Windows causó un commit con archivos en una carpeta incorrecta, corregido en el commit siguiente.

Mejoras para proyectos futuros:

- Reservar desde el inicio espacio de disco suficiente para Yocto (más de 100 GB) y documentar el entorno de construcción.
- Redactar los ADR al momento de tomar cada decisión y no después.
- Planificar la integración con el hardware real desde las primeras iteraciones y no solo al final.
- Verificar siempre en el hardware cualquier supuesto obtenido de documentación o de herramientas de IA antes de incorporarlo al repositorio.
- Documentar la polaridad de cada señal que atraviesa un optoacoplador o un transistor antes de escribir el código que la controla.
- Nombrar cada imagen con su versión y mostrarla al arrancar, y cambiar una sola cosa entre una imagen y la siguiente.
- Revisar qué periféricos del SoC comparte cada función (PWM, audio, SPI, I2S) al asignar los pines.

6. Conclusiones

Las estrategias más eficaces fueron las que produjeron evidencia verificable: la prueba incremental en el hardware y la depuración guiada por comandos de diagnóstico. Las menos eficaces dependieron de supuestos no verificados, ya fueran de la documentación, de los tutoriales o de los asistentes de IA. El principal aprendizaje del grupo es que, en sistemas embebidos, cada capa (hardware, kernel, imagen, biblioteca y servicio) debe validarse por separado en el dispositivo real antes de integrarla con la siguiente. En la integración final esto se tradujo en tres prácticas concretas: aislar la falla con una imagen mínima, comprobar el contenido de cada artefacto antes de grabarlo y cambiar una sola variable por prueba.